LAB 06

REGRESSION

POLYNOMIAL REGRESSION

Polynomial Regression is a form of regression analysis that models the relationship between the independent variable (X) and the dependent variable (Y) as an nth-degree polynomial.

Unlike simple or multiple linear regression, which assumes a straight-line relationship, polynomial regression can capture non-linear relationships by fitting a curved line to the data—for example, as X increases, Y might increase at an accelerating rate or follow a parabolic pattern.

While linear regression is limited to straight lines, polynomial regression extends this by adding higher-order terms (e.g., X^2, X^3) to better fit curved data patterns. It is still considered a type of linear regression in the machine learning sense because the coefficients are linear (the non-linearity comes from the powers of X).

Polynomial regression estimates how Y changes with X in a curved manner, allowing for more flexible modeling than simple linear regression. However, it risks overfitting if the degree is too high, where the model fits the training data too closely but performs poorly on new data.

With polynomial regression, we are trying to predict the Y variable from X using a polynomial relationship (i.e., a curve):

$$y = b_0 + b_1X + b_2X^2 + ... + b_nX^n$$

The symbol b_0 is known as the intercept (or constant), b_1 as the coefficient for the linear term, b_2 for the quadratic term, and so on up to b_n for the nth-degree term. b_0 is the value of the dependent variable when the independent variable is zero. b_1 tells the initial rate of change in the dependent variable with a unit change in the independent variable. · Higher coefficients (e.g., b_2) control the curvature, indicating acceleration or deceleration in the relationship.

The Y variable is known as the response or dependent variable since it depends on X. The X variable is known as the predictor or independent variable (and can be extended to multiple X variables for multivariate polynomial regression). The machine learning community tends to use other terms, calling Y the target and X a feature vector.

Key applications:

- Modeling growth rates (e.g., population growth that accelerates then plateaus)
- Predicting physical phenomena (e.g., trajectory of a projectile under gravity)
- Analyzing trends in data with curvature (e.g., car speed vs. fuel efficiency)

Polynomial Regression

```python
# Import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
```

```python
# Step 1: Create sample dataset
# Example: Plant height (cm) over weeks — shows non-linear (curved) growth
data = {
    'Week':     [1,  2,  3,  4,  5,  6,  7,  8,  9, 10],
    'Height_cm': [5, 12, 22, 35, 48, 60, 70, 78, 84, 88]
}

df = pd.DataFrame(data)
print("Plant Growth Dataset (non-linear pattern):")
print(df)
print()
```

```
Plant Growth Dataset (non-linear pattern):
   Week  Height_cm
0     1          5
1     2         12
2     3         22
3     4         35
4     5         48
5     6         60
6     7         70
7     8         78
8     9         84
9    10         88
```

```python
# Step 2: Prepare features (X) and target (y)
X = df[['Week']]          # input feature (only one for this example)
y = df['Height_cm']       # target

# Step 3: Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,          # 30% for testing
    random_state=42
)

print(f"Training samples: {len(X_train)}")
print(f"Testing samples:  {len(X_test)}")
print()
```

```
Training samples: 7
Testing samples:  3
```

```python
# Step 4: Create Polynomial Features (we'll use degree 2 = quadratic)
degree = 2
poly = PolynomialFeatures(degree=degree, include_bias=False)

# Transform training and test data to include x² terms
X_train_poly = poly.fit_transform(X_train)
X_test_poly  = poly.transform(X_test)
```

```python
# Step 5: Train a Linear Regression model on the polynomial features
model = LinearRegression()
model.fit(X_train_poly, y_train)
```

```
LinearRegression
LinearRegression()
```

```python
# Step 6: Show the polynomial equation (manually constructed)
coefficients = model.coef_
intercept = model.intercept_

print(f"Polynomial Regression Equation (degree {degree}):")
print(f"Height = {intercept:.2f}")
for i, coef in enumerate(coefficients, 1):
    print(f"      + {coef:.2f} × Week^{i}")
print()
```

```
Polynomial Regression Equation (degree 2):
Height = -10.99
      + 13.34 × Week^1
      + -0.32 × Week^2
```

```python
# Step 7: Make predictions on test set
y_test_pred = model.predict(X_test_poly)

# Show actual vs predicted
print("Test Set - Actual vs Predicted Height:")
test_results = pd.DataFrame({
    'Week': X_test['Week'].values.flatten(),
    'Actual_Height': y_test,
    'Predicted_Height': y_test_pred.round(1)
})
print(test_results.sort_values('Week'))
print()
```

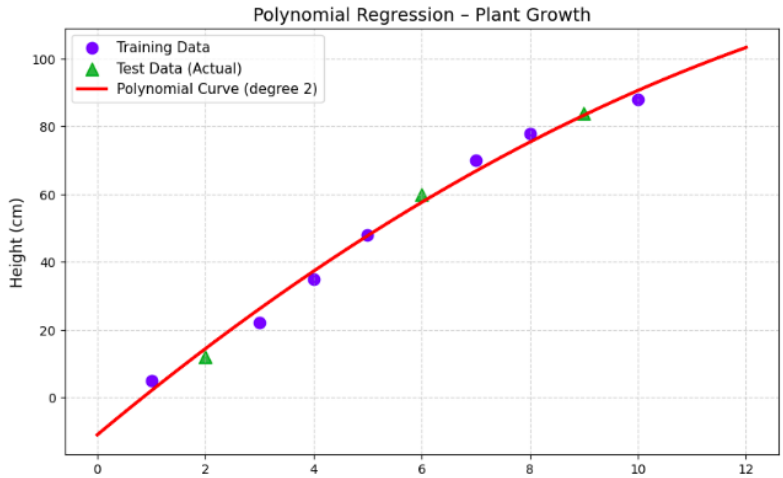
```
Test Set - Actual vs Predicted Height:
   Week  Actual_Height  Predicted_Height
1     2             12              14.4
5     6             60              57.6
8     9             84              83.3
```

```python
# Step 8: Predict for a new week
new_week = 11
new_week_poly = poly.transform([[new_week]])
predicted_height = model.predict(new_week_poly)[0]

print(f"Prediction for week {new_week}: {predicted_height:.1f} cm")
print()
```

```
Prediction for week 11: 97.3 cm

/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning:
  warnings.warn(
```

```python
# Step 9: Visualization - smooth curve
plt.figure(figsize=(10, 6))

# Training data
plt.scatter(X_train, y_train, color='blue', label='Training Data', s=80, alpha=0.7)

# Test data (actual)
plt.scatter(X_test, y_test, color='green', label='Test Data (Actual)', s=100, marker='^', alpha=0.7)

# Smooth curve for the whole range
X_curve = np.linspace(0, 12, 200).reshape(-1, 1)   # many points for smooth line
X_curve_poly = poly.transform(X_curve)
y_curve = model.predict(X_curve_poly)

plt.plot(X_curve, y_curve, color='red', linewidth=2.5, label=f'Polynomial Curve (degree {degree})')

plt.xlabel('Week', fontsize=12)
plt.ylabel('Height (cm)', fontsize=12)
plt.title('Polynomial Regression - Plant Growth', fontsize=14)
plt.legend(fontsize=11)
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature r
  warnings.warn(
```



KNN REGRESSION

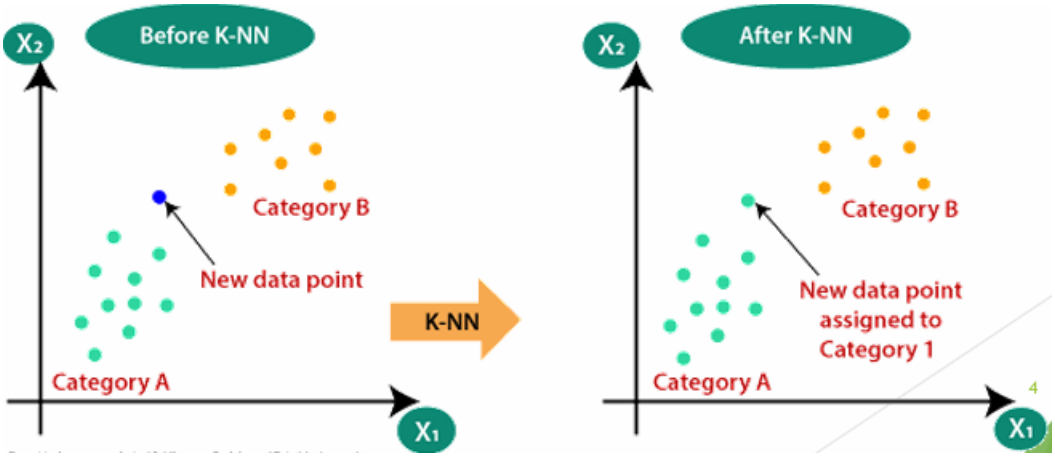
A supervised learning algorithm used for classification and regression. It is a lazy learning method (does not explicitly train a model). Predictions are made based on similarity to nearby data points. There is no model to be fit (as in regression). It means that KNN does not involve learning a model from training data in the way that regression or other machine learning models do.

The idea behind nearest neighbors is to select one or more examples from the training data to decide the predicted value for the sample at hand. The prediction results depend on how the features are scaled, how similarity is measured, and how big K is set. Also, all predictors must be in numeric form.

The idea behind K-Nearest Neighbours (KNN) is:

Find K records that have similar features (atleast 5 to 7)

- For classification, find out what the majority class is among those similar records and assign that class to the new record.
- For prediction, find the average among those similar records, and predict that average for the new record.

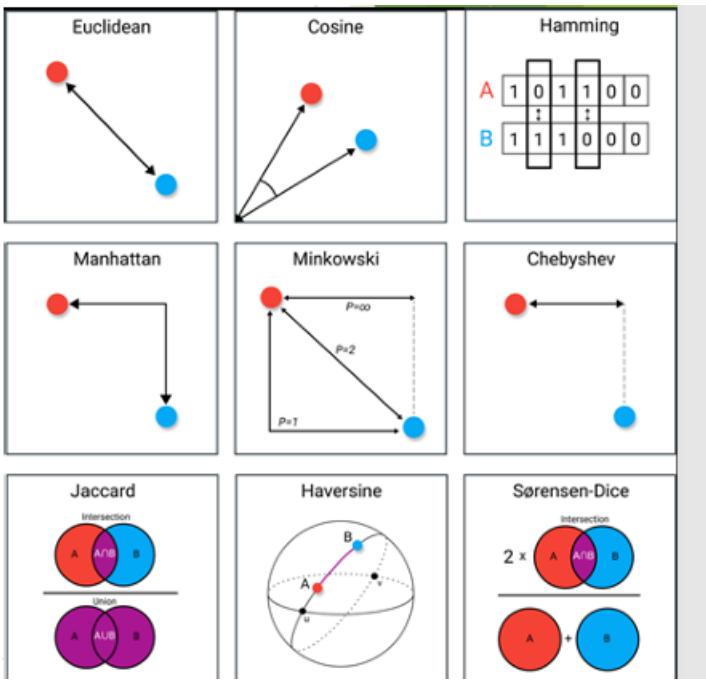


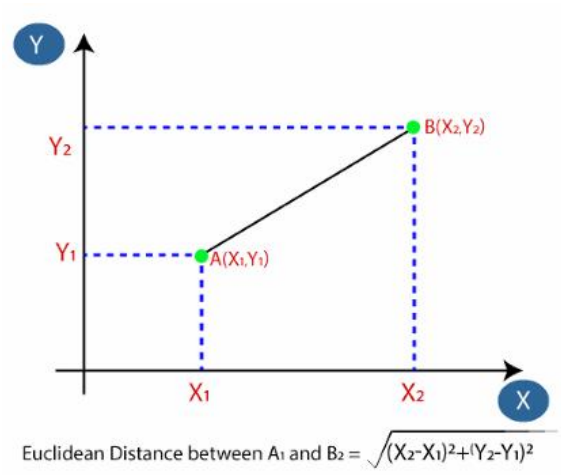
Similarity (nearness) is determined using a distance metric, which is a function that measures how far two records (x1, x2, …, xp) and (u1, u2, …, up) are from one another. The most popular distance metric between two vectors is Euclidean distance. To measure the Euclidean distance between two vectors, subtract one from the other, square the differences, sum them, and take the square root

$$\boxed{\sqrt{(x_1-u_1)^2+(x_2-u_2)^2+\cdots+(x_p-u_p)^2}.}$$

Another common distance metric for numeric data is Manhattan distance:

$$\boxed{|x_1-u_1|+|x_2-u_2|+\cdots+|x_p-u_p|}$$





Step 1: We Already Have Labeled Data

- The dataset contains pre-classified (labeled) data points.
- We know how many classesexist and which data points belong to which class.

Step 2: A New Data Point Comes In

- This new data point (test instance) does not have a label.
- Our goal is to determine which class it belongs to.

Step 3: Compute Distances

- KNN calculates the distance between the new point and all labeled points in the dataset.

Common distance metrics:

- Euclidean Distance (most common)
- Manhattan Distance
- Minkowski Distance

Step 4: Select the k Nearest Neighbors

- Sort all data points by distance.
- Pick the k closest points (smallest distances). υ

Step 5: Majority Voting (For Classification)

- Look at the k neighborsand see which class occurs most frequently.
- Assign the new point to the majority class.

The choice of K is very important to the performance of KNN. The simplest choice is to set K = 1, known as the 1-nearest neighbour classifier.

If K is too low, we maybe overfitting: including the noise in the data. Overfitting is an undesirable machine learning behavior that occurs when the machine learning model gives accurate predictions for training data but not for new data. Higher values of K provide smoothing that reduces the risk of overfitting in the training data.

If K is too high, we may oversmooth the data and miss out on KNN's ability to capture the local structure in the data, one of its main advantages.

The K that best balances between overfitting and oversmoothing is typically determined by accuracy metrics and in particular, accuracy with holdout or validation data. If you are using K and you have an even number of classes (e.g. 2) it is a good idea to choose a K value with an odd number to avoid a tie. And the inverse, use an even number for K when you have an odd number of classes

Task 1: Polynomial Regression (Curve Fitting)

Scenario: You are analyzing the relationship between **Engine Speed (RPM)** and **Fuel Consumption**. As RPM increases, fuel consumption doesn't just go up linearly; it curves upward sharply.

```
data = {
    'RPM': [1000, 1500, 2000, 2500, 3000, 3500, 4000, 4500, 5000, 5500],
    'Fuel_Consum': [5.2, 5.8, 6.5, 7.5, 9.0, 11.2, 14.5, 18.2, 23.0, 29.5]
}
```

Requirements:

1. Apply a **Degree 3** Polynomial transformation.
2. Calculate **MAE**, **RMSE**, and **R2 Score**.
3. **Visualization:** Plot the original data points and the smooth polynomial curve.
4. **Interpretation:** Why is a straight line a "poor fit" here? Look at the residuals at the high-end RPM values.

Task 2: KNN Regression (The Neighbourhood Logic)

Scenario: Predict a **Student's Final Grade** based on **Attendance (%)** and **Midterm Score**. Because these features have different scales (0-100 and 0-50), scaling is required.

```
data = {
    'Attendance': [95, 70, 45, 88, 60, 92, 50, 80, 75, 98],
    'Midterm': [48, 30, 15, 45, 20, 49, 10, 35, 32, 50],
    'Final_Grade': [90, 65, 30, 85, 45, 92, 25, 75, 70, 98]
}
```

Requirements:

1. **Scale** the features using StandardScaler.
2. Implement KNN Regressor with **K=3**.

3. **Experiment:** Change K to 1 and then to 9. Report which K gives the best R2 score.
4. **Logic Check:** Why does K=1 usually give a perfect score on Training data but fail on Test data?

Task 3: (Optional/Bonus) Manual Data Splitting

The Challenge: You are forbidden from using

- **from sklearn.model_selection import train_test_split.**

You must manually create X_train, X_test, y_train, and y_test.

Instructions for Students:

1. **Shuffle:** Use numpy.random.permutation or .sample(frac=1) in Pandas to mix the rows so the data isn't sorted.
2. **Calculate Split Point:** Find the index at 80% of the dataset length.
3. **Slice:** Use .iloc or slicing to divide the data into:
    - **Training Set:** First 80% of the rows.
    - **Testing Set:** Remaining 20% of the rows.
4. **Validation:** Run a Simple Linear Regression on your manual split and compare the results with built-in function.

Task 4: Real-World Application (Kaggle Dataset)

Requirement:

1. Go to **Kaggle** and find a regression dataset (e.g., *Medical Cost Personal Dataset*, *Real Estate Price Prediction*, or *Car Price Prediction*).

2. **Implementation:** Run both **Polynomial Regression** and **KNN Regression** on this dataset.

3. **Comparison:** Create a final table comparing the performance of all models on this real-world data.